

Trường ĐH Công Thương TP. HCM Khoa Công nghệ Thông tin Bộ môn Kỹ thuật phần mềm CÔNG NGHỆ .NET	BÀI 11. LẬP TRÌNH VỚI CƠ SỞ DỮ LIỆU (tiếp theo)	
--	--	--

A. MỤC TIÊU

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- Vận dụng các kỹ thuật kiểm tra dữ liệu (Validation) trong WPF.
- Thực hiện Validation bằng các kỹ thuật khác nhau.
- Sử dụng Trigger để thay đổi giao diện và cảnh báo dữ liệu không hợp lệ.
- Xây dựng chức năng điều hướng giữa các View theo MVVM.
- Thiết kế các chức năng quản lý dữ liệu đảm bảo tính hợp lệ và quy tắc nghiệp vụ.
- Rèn luyện kỹ năng phân tích yêu cầu và tổ chức mã nguồn ứng dụng.

B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SINH VIÊN

STT	Thiết bị - Phần mềm – Thư viện	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	
2	Microsoft Visual Studio 2019 trở lên (.NET Framework từ 4.7.2)	1	1	
3	Entity Framework (6.x.x)	1	1	
4	Microsoft SQL Server từ 2017	1	1	

C. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

Chương 4. Lập trình với cơ sở dữ liệu (tiếp theo)

4.4. Validation và xử lý ngoại lệ

4.4.1. Validation dữ liệu

Validation là quá trình kiểm tra dữ liệu đầu vào trước khi cho phép lưu xuống cơ sở dữ liệu nhằm đảm bảo dữ liệu tuân thủ quy tắc nghiệp vụ, hạn chế lỗi và tăng độ tin cậy của hệ thống.

Trong ứng dụng WPF theo mô hình MVVM, Validation nên đặt chủ yếu ở ViewModel hoặc Model; View chỉ đảm nhiệm việc hiển thị trạng thái lỗi. Một số kỹ thuật Validation phổ biến:

a) Validation bằng thuộc tính trong ViewModel

Kiểm tra dữ liệu ngay khi người dùng nhập (ví dụ: rỗng, sai định dạng, giá trị không hợp lệ).

Ví dụ:

```
public string MaSV
{
    get => _maSV;
    set {
        if (string.IsNullOrEmpty(value))
            throw new Exception("Mã sinh viên không được để trống");
        _maSV = value;
        OnPropertyChanged();
    }
}
```

Ưu điểm: đơn giản, dễ hiểu.

Hạn chế: nếu lạm dụng exception để kiểm tra liên tục sẽ khó mở rộng và khó hiển thị lỗi thân thiện trên UI.

b) ValidationRule trong XAML

Kiểm tra dữ liệu trực tiếp tại giao diện, giúp tách biệt xử lý Validation khỏi ViewModel. Thích hợp cho các luật đơn giản và độc lập với dữ liệu nghiệp vụ, ví dụ kiểm tra tuổi, kiểm tra số, giới hạn khoảng giá trị.

Ví dụ:

```
<TextBox>
    <TextBox.Text>
        <Binding Path = "Tuoi" UpdateSourceTrigger="PropertyChanged">
        <Binding.ValidationRules>
            <local:TuaiValidationRule />
```

```
</Binding.ValidationRules>
    </Binding>
</TextBox.Text>
</TextBox>
```

c) Validation bằng DataAnnotations

Sử dụng các thuộc tính mô tả ràng buộc trên model hoặc lớp nhập liệu riêng như [Required], [StringLength], [Range].

```
[Required(ErrorMessage = "Tên lớp không được để trống")]
[StringLength(50)]
public string TenLop { get; set; }
```

4.4.2. Xử lý dữ liệu với IDataErrorInfo

Cung cấp thông báo lỗi theo từng thuộc tính. WPF sẽ tự động lấy thông báo lỗi từ ViewModel để hiển thị trực tiếp trên điều khiển nhập liệu.

a) Các thành phần chính

this[string columnName]: trả về lỗi của từng thuộc tính.

Error: trả về lỗi tổng quát của đối tượng (thường để null nếu không dùng).

b) Quy trình xử lý

Bước 1: Người dùng nhập dữ liệu tại View.

Bước 2: Binding gọi thuộc tính **this[columnName]** trong ViewModel

Bước 3: Nếu có lỗi, WPF hiển thị lỗi và đánh dấu điều khiển tương ứng.

Bước 4: Nếu không có lỗi, dữ liệu được chấp nhận để lưu.

Ví dụ:

```
public class SinhVienViewModel : IDataErrorInfo
{
    public string HoTen { get; set; }
    public string MaSV { get; set; }
```

```

public string Error => null;
public string this[string columnName] {
    get {
        switch (columnName) {
            case "MaSV":
                if (string.IsNullOrEmpty(MaSV))
                    return "Mã sinh viên không được để trống";
                break;

            case "HoTen":
                if (string.IsNullOrEmpty(HoTen))
                    return "Họ tên không được để trống";
                break;
        }
        return null;
    }
}

public bool IsValid =>
    string.IsNullOrEmpty(this[nameof(MaSV)]) &&
    string.IsNullOrEmpty(this[nameof(HoTen)]);
}

```

Kết nối với giao diện:

```

<TextBox Text = "{Binding MaSV,
    ValidatesOnDataErrors=True,
    UpdateSourceTrigger=PropertyChanged,
    NotifyOnValidationError=True }"
    ToolTip = "{Binding (Validation.Errors)[0].ErrorContent,
    RelativeSource={RelativeSource Self}}"
/>

```

4.4.3. Trigger

Trigger cho phép thay đổi giao diện theo trạng thái dữ liệu mà không cần viết code-behind. Đây là kỹ thuật rất phù hợp khi kết hợp với MVVM.

- Property Trigger: Kích hoạt khi thuộc tính giao diện thay đổi.
- Data Trigger: Kích hoạt khi giá trị Binding thỏa điều kiện (ví dụ: bật/tắt nút Lưu).
- Event Trigger: Kích hoạt khi xảy ra một sự kiện.

a) PropertyTrigger: Kích hoạt khi giá trị của một thuộc tính thay đổi.

Ví dụ: đánh dấu màu đỏ khi TextBox có lỗi

```
<TextBox Text = "{Binding MaSV, ValidatesOnDataErrors=True}" >
  < TextBox.Style >
    < Style TargetType="TextBox">
      <Style.Triggers>
        <Trigger Property = "Validation.HasError" Value="True">
          <Setter Property = "BorderBrush" Value="Red"/>
          <Setter Property = "ToolTip"
            Value="{Binding RelativeSource={RelativeSource Self},
              Path=(Validation.Errors)[0].ErrorContent
            }"/>
        </Trigger>
      </Style.Triggers>
    </Style>
  </TextBox.Style>
</TextBox>
```

b) DataTrigger: Kích hoạt khi giá trị Binding đạt điều kiện.

Ví dụ: bật nút Lưu khi dữ liệu hợp lệ

```
<Button Content = "Lưu" >
  < Button.Style >
    < Style TargetType="Button">
      <Setter Property = "IsEnabled" Value="False"/>
```

```

<Style.Triggers>
    <DataTrigger Binding = "{Binding IsValid}" Value="True">
        <Setter Property = "IsEnabled" Value="True"/>
    </DataTrigger>
</Style.Triggers>
</Style>
</Button.Style>
</Button>

```

c) EventTrigger: Kích hoạt khi có sự kiện (ít dùng trong MVVM)

2. Bài tập thực hành trên lớp

Bài tập có hướng dẫn

Bài 1. Ứng dụng quản lý Khoa có giao diện như hình sau (sử dụng bài tập hướng dẫn 1 trong bài thực hành 09)

Yêu cầu: thực hiện kiểm tra dữ liệu (Validation bằng IDataErrorInfo)

Trước khi thao tác thêm hoặc sửa, phải kiểm tra:

- Đối với Mã khoa: Không được để trống, Không vượt quá 5 ký tự, Không trùng với mã khoa đã có khi thêm mới
- Đối với Tên khoa: Không được để trống, Tối đa 50 ký tự

Hướng dẫn thực hiện:

Bước 1: Tạo lớp nhập liệu **KhoaInputViewModel.cs** kế thừa **BaseViewModel** và thực thi interface **IDataErrorInfo**

ViewModels → Add → Class → KhoaInputViewModel.cs

```

internal class KhoalInputViewModel : BaseViewModel, IDataErrorInfo
{
    private bool _IsEdit = false;
    public bool IsEdit
    {
        get => _IsEdit;
        set
        {
            _IsEdit = value;
            OnPropertyChanged(nameof(IsEdit));
        }
    }
    private string _MaKhoa;
    public string MaKhoa {
        get => _MaKhoa;
        set {
            _MaKhoa = value;
            OnPropertyChanged(nameof(MaKhoa));
            OnPropertyChanged(nameof(IsValid));
        }
    }
    private string _TenKhoa;
    public string TenKhoa {
        get => _TenKhoa;
        set {
            _TenKhoa = value;
            OnPropertyChanged(nameof(TenKhoa));
            OnPropertyChanged(nameof(IsValid));
        }
    }
    public string Error => null;
}

```

```

//Validation từng thuộc tính
public string this[string columnName]
{
    get {
        if (columnName == nameof(MaKhoa))
        {
            if (string.IsNullOrEmpty(MaKhoa))
                return "Mã khoa không được để trống";
            if (MaKhoa.Length > 5)
                return "Mã khoa tối đa 5 ký tự";
        }
        if (columnName == nameof(TenKhoa)) {
            if (string.IsNullOrEmpty(TenKhoa))
                return "Tên khoa không được để trống";
            if (TenKhoa.Length > 50)
                return "Tên khoa tối đa 50 ký tự";
        }
        return null;
    }
}
//Kiểm tra toàn form
public bool IsValid =>
    string.IsNullOrEmpty(this[nameof(MaKhoa)]) &&
    string.IsNullOrEmpty(this[nameof(TenKhoa)]);
}

```

Bước 2: Kết nối **KhoaInputViewModel** với **KhoaViewModel**

- Khai báo biến nhập liệu (mở KhoaViewModel.cs):

```

public KhoaInputViewModel NewKhoa { get; set; }
= new KhoaInputViewModel();

```

- Thuộc tính SelectedKhoa:

```

public Khoa SelectedKhoa
{
    get => _SelectedKhoa;
}

```



```

set {
    _SelectedKhoa = value;
    OnPropertyChanged(nameof(SelectedKhoa));
    if (SelectedKhoa != null) {
        NewKhoa.IsEdit = true;
        NewKhoa.MaKhoa = SelectedKhoa.MaKhoa;
        NewKhoa.TenKhoa = SelectedKhoa.TenKhoa;
    }
    else
        NewKhoa.IsEdit = false;
}
}

```

Bước 3: Bật Validation trên giao diện (QLKhoaView.xaml)

- Textbox MaKhoa

```

<TextBox Text="{Binding NewKhoa.MaKhoa,
    UpdateSourceTrigger=PropertyChanged,
    ValidatesOnDataErrors=True,
    NotifyOnValidationError=True}"
    ToolTip="{Binding RelativeSource={RelativeSource Self},
    Path=(Validation.Errors)[0].ErrorContent}" />

```

- Textbox TenKhoa: tương tự MaKhoa

Bước 4: Kiểm tra khi bấm nút Thêm

Trong KhoaViewModel.cs

```

void Add() {
    if (!NewKhoa.IsValid) {
        MessageBox.Show(
            "Dữ liệu không hợp lệ.\nKiểm tra lại các trường.", "Thông báo",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }
}

```

```

//KIỂM TRA TRÙNG MÃ KHOA
bool isDuplicate = db.Khoas.Any(k => k.MaKhoa ==
                                                NewKhoa.MaKhoa);

if (isDuplicate) {
    MessageBox.Show( "Mã khoa đã tồn tại!", "Lỗi",
                    MessageBoxButton.OK, MessageBoxImage.Error);

    return;
}
Khoa khoa = new Khoa {
    MaKhoa = NewKhoa.MaKhoa,
    TenKhoa = NewKhoa.TenKhoa
};
db.Khoas.Add(khoa);
db.SaveChanges();
DS_Khoa.Add(khoa);
SelectedKhoa = khoa;
MessageBox.Show("Thêm Khoa thành công!", "Thông báo");
NewKhoa.IsEdit = false;
NewKhoa.MaKhoa = SelectedKhoa.MaKhoa;
NewKhoa.TenKhoa = SelectedKhoa.TenKhoa;
}

```

Bước 5: Kiểm tra khi bấm nút “Sửa”

```

void Update()
{
    if (!NewKhoa.IsValid)
    {
        MessageBox.Show( "Dữ liệu cập nhật không hợp lệ.",
                        "Thông báo", MessageBoxButton.OK,
                        MessageBoxImage.Warning);
        return;
    }
}

```

```

SelectedKhoa.MaKhoa = NewKhoa.MaKhoa;
SelectedKhoa.TenKhoa = NewKhoa.TenKhoa;

db.SaveChanges();
LoadData();
MessageBox.Show("Cập nhật thành công!", "Thông báo");
}

```

🔍 Lưu ý: khi sửa, có thể khóa TextBox Mã khoa để tránh thay đổi khóa chính.

Bài 2. Ứng dụng WPF Quản lý lớp có giao diện như sau

Yêu cầu: sử dụng Trigger trong XAML để làm giao diện thân thiện hơn.

- Khi TextBox Mã lớp trống: nền màu xanh nhạt.
- Khi chưa nhập Mã lớp: xuất hiện Tooltip cảnh báo.
- Khi DataGrid chưa có dữ liệu: hiển thị thông báo “Chưa có dữ liệu lớp”.

Hướng dẫn:

1. Khi TextBox **Mã lớp trống** → nền **màu xanh nhạt** (Nhắc người dùng nhập dữ liệu bắt buộc).

```

<TextBox x:Name="txt_malop" Height="28" Margin="0 4 0 12">
    <TextBox.Style>
        <Style TargetType="TextBox">

```

```

<Style.Triggers>
  <Trigger Property="Text" Value="">
    <Setter Property="Background" Value="AliceBlue"/>
  </Trigger>
</Style.Triggers>
</Style>
</TextBox.Style>
</TextBox>

```

2. Khi chưa nhập Mã lớp xuất hiện Tooltip cảnh báo

```

<TextBox x:Name="txt_malop" Height="28" Margin="0 4 0 12">
  <TextBox.Style>
    <Style TargetType="TextBox">
      <Style.Triggers>
        <Trigger Property="Text" Value="">
          <Setter Property="ToolTip" Value="Mã lớp không trống"/>
        </Trigger>
      </Style.Triggers>
    </Style>
  </TextBox.Style>
</TextBox>

```

3. Khi DataGridView chưa được load hiển thị “Chưa có dữ liệu lớp”

Hướng dẫn:

```
<Grid>
  <DataGrid x:Name="dgLop" />
  <TextBlock Text="Chưa có dữ liệu lớp"
    Foreground="Gray"
    FontStyle="Italic"
    HorizontalAlignment="Center"
    VerticalAlignment="Center">
    <TextBlock.Style>
      <Style TargetType="TextBlock">
        <Setter Property="Visibility" Value="Collapsed"/>
        <Style.Triggers>
          <!--Khi số dòng hiện tại trong DataGrid bằng 0-->
          <!--Hiện TextBlock-->
            <DataTrigger Binding="{Binding ElementName=dgLop,
                                   Path=Items.Count}"
                          Value="0">
              <Setter Property="Visibility" Value="Visible"/>
            </DataTrigger>
          </Style.Triggers>
        </Style>
      </TextBlock.Style>
    </TextBlock>
  </Grid>
```

Bài tập sinh viên tự làm

Bài 3. Ứng dụng WPF Quản lý môn học có giao diện như sau:

Quản lý Môn học

Mã môn: Tên môn học:

Số TC: Tính chất:

Mã môn	Tên môn học	Số TC	Tính chất
--------	-------------	-------	-----------

Yêu cầu:

Trước khi thực hiện các thao tác Thêm hoặc Cập nhật dữ liệu môn học, hệ thống phải thực hiện kiểm tra dữ liệu đầu vào với các điều kiện sau:

- Đối với Mã môn
 - Không được để trống.
 - Độ dài không vượt quá quy định của CSDL.
 - Khi thêm mới, mã môn không được trùng với mã môn đã tồn tại trong hệ thống.
- Đối với Tên môn học
 - Không được để trống.
 - Độ dài không vượt quá quy định cho phép.
 - Tên môn học là duy nhất
- Đối với Số tín chỉ
 - Không được để trống.
 - Phải là số nguyên dương.
 - Giá trị số tín chỉ phải nằm trong khoảng hợp lệ theo quy định đào tạo.
- Nguyên tắc xử lý

- Nếu dữ liệu không hợp lệ, hệ thống không cho phép cập nhật dữ liệu vào CSDL.
- Hiện thị thông báo lỗi để người dùng biết và chỉnh sửa lại dữ liệu.
- Chỉ thực hiện lưu dữ liệu khi tất cả các điều kiện đều hợp lệ.

3. Bài tập về nhà

Xây dựng giao diện quản lý sinh viên như sau:

The screenshot shows a web application titled "Quản lý sinh viên" (Student Management). Below the title is a subtitle "Thêm – chỉnh sửa – quản lý thông tin sinh viên". The interface is divided into two main sections. On the left, there is a form titled "Thông tin sinh viên" (Student Information) with the following fields: "Mã sinh viên" (Student ID) with a text input, "Họ tên" (Full Name) with a text input, "Giới tính" (Gender) with radio buttons for "Nam" (Male) and "Nữ" (Female), "Ngày sinh" (Date of Birth) with a date picker showing "Select a date" and "15", and "Lớp" (Class) with a dropdown menu. At the bottom of the form are three buttons: "+ Thêm" (Add), "Chỉnh sửa" (Edit), and "Lưu" (Save). On the right, there is a table with columns: "Mã SV", "Họ tên", "Giới tính", "Ngày sinh", and "Lớp". The table body is currently empty.

Yêu cầu:

Thực hiện kiểm tra dữ liệu đầu vào trước khi cho phép lưu:

- Mã sinh viên:
 - Không được để trống.
 - Độ dài tối đa: 10 ký tự.
 - Không được trùng với mã sinh viên đã tồn tại khi thêm mới.
 - Khi sửa, không cho phép thay đổi mã sinh viên.
- Họ tên
 - Không được để trống.
 - Độ dài tối đa: 50 ký tự.
- Giới tính
 - Bắt buộc phải chọn.

- Ngày sinh
 - Bắt buộc nhập.
 - Sinh viên phải từ 18 tuổi trở lên.
- Mã lớp
 - Bắt buộc chọn từ danh sách lớp (ComboBox).

🔍 Lưu ý: Nếu dữ liệu không hợp lệ:

- Không cho phép lưu.
- Hiện thị thông báo lỗi trực tiếp trên giao diện.
- Nút Lưu chỉ được bật khi dữ liệu hợp lệ.
- Áp dụng IDataErrorInfo để Validation dữ liệu.
- Sử dụng Trigger hoặc DataTrigger để:
 - Đổi màu ô nhập khi có lỗi.
 - Bật/ Tắt nút chức năng theo trạng thái.